

```
#!/usr/bin/env python3
```

```
"""
```

AESOP Memory Platform Module

Source Origin: PROPOSED WORKFLOWS AND FILE STRUCTURES / aesop_mg.py

Target Subsystem: _dumbass_universal_memory / mem0 & data_vault/04_skills_runtime

Implements the standalone state persistence layer for local agents, managing session parameters, transactional state commits, and history rollbacks.

```
"""
```

```
import json
```

```
import os
```

```
import time
```

```
from typing import Dict, List, Any, Optional
```

```
class AesopPlatformMemory:
```

```
    def __init__(self, storage_path: str = "aesop_platform_memory.json", platform_tier: str = "AESOP-XI"):
```

```
        self.storage_path = storage_path
```

```
        self.platform_tier = platform_tier
```

```
        self._ensure_storage()
```

```
    def _ensure_storage(self):
```

```
        if not os.path.exists(self.storage_path):
```

```
            initial_state = {
```

```
                "system_tier": self.platform_tier,
```

```
                "created_at": time.time(),
```

```
                "history": [],
```

```
                "active_locks": []
```

```
            }
```

```
            with open(self.storage_path, "w", encoding="utf-8") as f:
```

```
                json.dump(initial_state, f, indent=2)
```

```
    def retrieve_context(self, prompt: str, max_lookback: int = 3) -> str:
```

```
        """Pulls bounded recent conversational state without flooding prompt tokens."""
```

```
        try:
```

```
            with open(self.storage_path, "r", encoding="utf-8") as f:
```

```
                data = json.load(f)
```

```
            history = data.get("history", [])
```

```
            if not history:
```

```
                return f"Active Platform Layer: {self.platform_tier}. Clean state initialization."
```

```
            recent = history[-max_lookback:]
```

```
            summary = "; ".join([f"Q: {h.get('prompt')[:40]} -> A: {h.get('output')[:60]}" for h in recent])
```

```

        return f"Active Platform Layer: {self.platform_tier}. Recent State: {summary}"
    except Exception as e:
        return f"Active Platform Layer: {self.platform_tier}. Fallback context."

def commit_state(self, prompt: str, output: str, metadata: Optional[Dict[str, Any]] = None):
    """Atomically appends execution outcome and updates state ledger."""
    try:
        with open(self.storage_path, "r+", encoding="utf-8") as f:
            data = json.load(f)
            entry = {
                "timestamp": time.time(),
                "prompt": prompt,
                "output": output,
                "metadata": metadata or {}
            }
            data.setdefault("history", []).append(entry)
            f.seek(0)
            json.dump(data, f, indent=2)
            f.truncate()
    except Exception as e:
        pass

if __name__ == "__main__":
    mem = AesopPlatformMemory()
    print("Memory initialized. Context:", mem.retrieve_context("boot"))

```